

In C, arguments can be passed to functions primarily in two ways: call by value and call by reference (achieved using pointers). In call by value, a copy of the actual argument is passed to the function, so any changes made inside the function do not affect the original variable; this approach is simple, safe, and prevents unintended side effects, but it can be inefficient when dealing with large data structures because it involves copying data. On the other hand, call by reference is implemented by passing the address of a variable using pointers, allowing the function to directly access and modify the original data; this method is more efficient in terms of memory and performance, especially for large arrays or structures, and it enables functions to return multiple values. However, it also introduces risks such as unintended modifications, increased complexity, and potential errors like dereferencing invalid pointers if not handled carefully. Additionally, arrays in C are inherently passed as pointers, meaning they naturally follow reference-like behavior even when used in function arguments. Overall, the choice between these methods depends on the need for data protection versus performance and flexibility, with each approach offering its own advantages and limitations in program design.